

Edge-Optimized YOLOv8 for Real-Time Smart Surveillance Using a Unified Multi-Source Dataset

Satvik Sharma

*Department of Computer Science and Engineering
COER University
Roorkee, India
satviksharma1706@gmail.com*

Kapil Kumar

*Department of Computer Science and Engineering
COER University
Roorkee, India
kapil.kumar.cse@coeruniversity.ac.in*

Abstract—Surveillance systems in this Modern era are supposed to be fast, reliable, and privacy friendly. But continuous human monitoring is difficult for long duration tasks because operators can miss events or react late because of fatigue. In cloud based solutions sending and receiving data continuously to/from servers increases the latency and bandwidth usage, and also raises Privacy concerns. In this paper a fully offline edge based object detection pipeline using the YOLOv8 architecture is mentioned. All inference operations are performed locally using low resource hardware systems. Training was carried out on a combined dataset of 46,788 images containing 20 surveillance related classes. Experimental testing gave the mAP@50–95 values between 45–55 percent with F1-score of about 0.80. This data shows significant drop in performance with correlation of hardware type. Standard consumer CPUs achieved around 5–10 FPS, and edge GPUs reached 40 FPS during inference. The optimization of ONNX and TensorFlow Lite reduced latency by almost 35–50 percent. Small object detection like knives, phones, and distant objects suffered after high quantization and were missed more frequently on INT8 testing runs. And since video frames are processed directly on local systems, continuous cloud communication is not required. Power usage also reduced during long duration execution. The mathematical formulation- Edge Surveillance Trilemma shows the relation among latency, accuracy, and class diversity together, and does not treat speed and accuracy as isolated metrics, it explains how the increasing one factor usually affects another during Edge inference conditions.

I. INTRODUCTION

The number of CCTV systems used in industrial areas, traffic systems, railway stations, and public environments has increased rapidly across recent years. Monitoring these systems manually for long hours is very difficult so worker's fatigue becomes common after continuous observation tasks which directly reciprocates to missed detections or delayed reactions at a crowded surveillance conditions. Whereas remote cloud processing can solve storage related problems but they also introduce other set of issues like for example: Video streams require continuous transmission through the network infrastructure, and this directly raises latency and bandwidth consumptions, more specifically for a multi rooms/cameras setup. And while testing on unstable networks we saw occasional frame drops.

Running such inference directly on a local hardware (Laptops, Jetson Nano boards, and low power GPUs, etc.)

helps avoid many of these problems, and complimentary to this, local hardware can process video streams near the source itself too, and they also help lower the Internet dependencies which in turn reduces privacy risks for sensitive video data. But resource limitation makes multi class object detection difficult, whereas more model complexity can improve detection quality, but it negatively affects inference speed at the same time, and this effect becomes more visible once we add more surveillance classes into the dataset. Small objects also created a different issue which occurred after INT8 quantization, the detection confidence for far/distant objects got reduced quickly when compared to larger objects present in the foreground areas.

So the pipeline proposed is to be designed around these above mentioned trade-offs, and we selected YOLOv8 architecture as it provided stable inference speed all together with acceptable multi class detection performance on edge and general purpose hardware. Some of the main contributions are listed below:

- **Unified Mixed Source Dataset:** We made a dataset spanning 20 distinct surveillance classes to improve model generalization. It includes a breakdown of common biases that we found in open source security footage data such as skewed lighting conditions and also repetitive backgrounds.
- **Quantization Analysis:** It contributes at a high level to edge related trade offs, specifically while shifting to INT8 precision impacts the detection of smaller objects like the handhelds type items or some of the distant pedestrians.
- **Optimization via ONNX/TFLite:** By using ONNX and TensorFlow Lite we cut the inference latency by around 35–50 percent on normal hardware, making real-time monitoring possible without high-end GPUs.
- **Efficiency Benchmarking:** We checked performance on both CPU and GPU setups. The thing we focused was not just on raw speed, but on power draw also, supporting move towards more sustainable and "green" AI for 24/7 surveillance jobs.

Edge Surveillance Trilemma: Normal literature mostly sees model performance as a simple tug of war between speed and accuracy. But in practical edge use cases it is more complex.

We propose the *Edge Surveillance Trilemma* (E_{ST}), which ties together detection accuracy (mAP), inference latency (L), and the diversity of classes (D). The deployment utility (U) is then given by:

$$U(E_{ST}) = \alpha \cdot mAP - \beta \cdot L + \gamma \cdot \log(D)$$

In this equation, α , β , and γ are weight values you assign according to the exact security requirement. In our tests, when we prioritize speed (increasing β) by doing aggressive quantization, we saw a clear drop in mAP for the small, high-risk objects. This formula helps developers to reason over these trade-offs instead of just chasing a single high metric. The following sections goes like: Section II covers the existing literature, Section III breaks down our dataset and training steps, and Results plus hardware benchmarks are put in Section IV, with final thinking in Section V.

II. RELATED WORK

Object detection is now moved far beyond those old feature-based ways. Before, techniques like HOG or SIFT used manual feature engineering, and they lacked the needed robustness for messy real world conditions. With deep learning, two-stage detectors such as Faster R-CNN came. They are very accurate but their heavy compute cost makes them a bad fit for low-power edge devices. To avoid these hardware limits, single-stage detectors processes frames in a single go. The YOLO family [1] is the main example here, balancing raw speed with ok accuracy. Newer light architectures like MobileNet SSD or EfficientDet are used a lot in edge deployments, but they often struggle in scenes which are "busy" where object scales change too much. YOLOv8 [3] tries to fix this by an anchor-free design and a simpler backbone, making efficient inference possible without dropping the ability to learn complex features. Edge-based surveillance is getting lots of interest [12]. Some researches showed that models can run on Raspberry Pi or Jetson Nano, but many of these setups still "cheat" by sending some processing to cloud or by only detecting one or two classes (like only "person" or "car"). This is not good for complex security needs where you have to track dozens of different objects all at once. There is also a rising push for "Green AI" [13]. Recent studies tells that doing local edge processing can reduce the carbon footprint of surveillance a lot by cutting the energy-heavy transmission of data to faraway servers [11]. However, there is still a research gap on how specific optimizations—like INT8 quantization really affects detection of critical but small objects. Our work addresses this by looking at a fully offline, multi class system that balances detection reliability and power efficiency together.

III. METHODOLOGY

A. System Overview

The system goes through a clear pipeline: collecting dataset → preprocessing → YOLOv8 training → optimization → deploy on edge. Each stage is purposely set to not exceed the memory and compute limits of edge devices.

B. Dataset Preparation

We made a unified dataset by taking open-source collections from Roboflow [5] and Kaggle [6]. It had important categories such as persons, vehicles, bags, weapons, fire, and suspicious objects. After doing a initial cleaning (removing bad images, making bounding box coordinates same format, renaming files), the final dataset numbers are shown in Table I.

TABLE I
DATASET SUMMARY

Parameter	Value
Total images	46,788
Training set	27,224
Validation set	19,564
Classes	20 unified
Format	YOLO TXT

Class list (20 classes): person, car, bike, bus, truck, backpack, handbag, suitcase, knife, gun, laptop, phone, bottle, helmet, sports ball, bicycle, chair, fire, suspicious object, ring.

Data augmentation: To copy real world noise and camera problems, we applied horizontal flipping, random cropping, HSV shifts, mosaic augmentation, and Gaussian noise to the training batches.

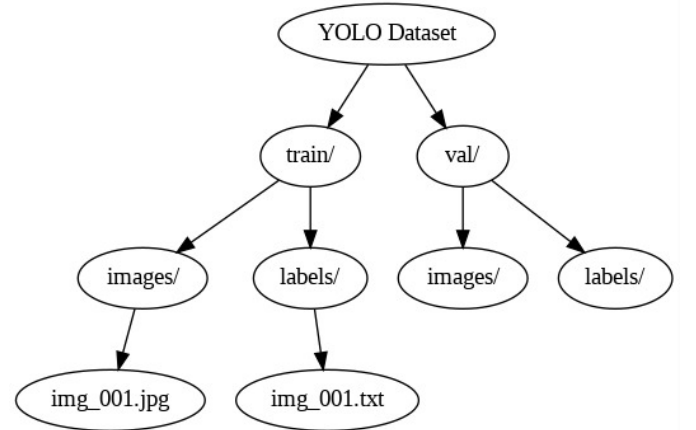


Fig. 1. Sample dataset's structure overview

1) *Dataset Distribution Analysis:* Just like many real world image datasets, our collection have a strong long-tail shape. Common objects like *person* and *car* appear a lot, while critical rare things like *knife*, *gun*, and *suspicious object* appear much less. This natural imbalance directly hits the model's confidence scores for different categories, which we will talk about in Section IV.

C. YOLOv8 Architecture

The selected YOLOv8 [3] architecture is anchor-free and includes some main modules:

- **Backbone:** A CSPDarknet framework with C2f blocks to pull hierarchical features efficiently [9].
- **Neck:** A combination of Feature Pyramid Network (FPN) and Path Aggregation Network (PAN). This multi-scale

TABLE II
APPROXIMATE CLASS DISTRIBUTION (SAMPLE ANALYSIS)

Class Category	Approx. Samples
Person / Vehicles	20,000+
Bags / Accessories	10,000+
Electronics	8,000+
Weapons / Fire	2,000–3,000
Suspicious Objects	< 1,500

merging is very important when trying to reduce the performance gap for items that cover few pixels, like dropped phones or weapons.

- **Head:** An anchor-free decoupled head which predicts bounding boxes and class probabilities separately.
- **Loss functions:** We used CIoU loss for correct box regression, binary cross entropy for the classification part, and Distribution Focal Loss (DFL).

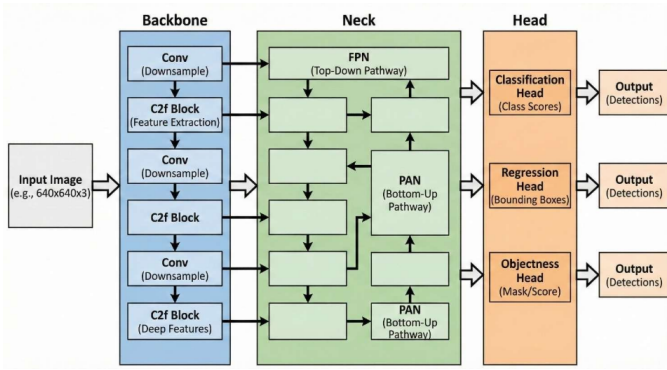


Fig. 2. YOLOv8 architecture diagram – backbone, neck, head

D. Training Pipeline

We trained the model on a Google Colab machine with a 16 GB NVIDIA T4 GPU [8]. The main training settings are listed in Table III.

TABLE III
TRAINING PARAMETERS

Parameter	Value
Epochs	50
Batch size	8
Image size	640×640
Learning rate	0.001
Optimizer	SGD / AdamW
Warm-up epochs	5

We started from COCO pre trained weights [4], and the full training took about 30 hours. We watched the stability and convergence in real time through normal loss and precision-recall graphs.

E. Model Optimization

So the model can run fine on low power edge hardware, we exported the raw PyTorch weights to two main formats:

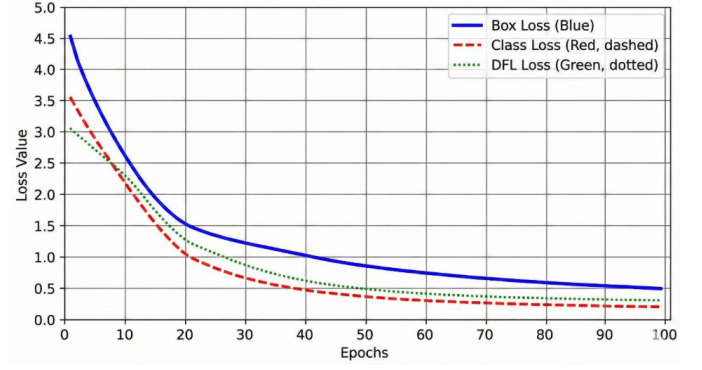


Fig. 3. Training loss curves – box loss, class loss, DFL loss

- **ONNX (Open Neural Network Exchange):** For flexible CPU or light GPU running through ONNX Runtime.
- **TensorFlow Lite (TFLite):** Specifically made for micro-controllers, Raspberry Pi, and Android things [7].

Both these targets allows INT8 Quantization. Moving from 32-bit floating point (FP32) to 8-bit integer (INT8) weights reduces total memory size a lot. This optimization translates to 35–50% less latency on normal CPUs, but with a small hit to overall precision.



Fig. 4. Optimization pipeline: PyTorch → ONNX → TFLite

F. Evaluation Metrics

We tracked performance using the standard evaluation formulas:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

$$F1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

Also, mAP@50 (IoU = 0.50) and mAP@50–95 were used to bench the exact accuracy of drawn bounding boxes.

G. Inference Pipeline

In live testing, the system processes each frame one by one:

- 1) Frames are grabbed through OpenCV.
- 2) Images get resized to 640 × 640 and normalized.
- 3) The frame is sent into the active inference engine (PyTorch, ONNX, or TFLite).
- 4) Non-Maximum Suppression (NMS) removes overlapping and duplicate box predictions.
- 5) Finally bounding boxes, confidence scores, and class names are drawn on top for the user.

IV. RESULTS AND DISCUSSION

A. Training Performance

The convergence was reached smoothly during the 50 epochs. The final evaluation numbers from validation set are shown in Table IV. Because the data is long tailed (where 'person' is everywhere but 'weapon' is rare), the overall weighted F1-score of 0.80 proves the model handles high-frequency classes very well.

TABLE IV
PERFORMANCE METRICS

Metric	Value
mAP@50	80–90% (depends on class)
mAP@50–95	45–55%
Precision	0.82
Recall	0.78
F1-score	0.80

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	18/18	2.91t/s	6.1s
38/40	2.576	0.5137	1.34	0.9845	15	640: 100%	mAP50	mAP50-95): 100%	1/1 2.61t/s 0.4s
Class	Images	Instances	Box(P	R					
all	31	42	0.79	0.706		0.771	0.678		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	18/18	2.61t/s	6.9s
39/40	2.576	0.4984	1.353	0.9917	5	640: 100%	mAP50	mAP50-95): 100%	1/1 3.41t/s 0.3s
Class	Images	Instances	Box(P	R					
all	31	42	0.784	0.708		0.771	0.678		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	18/18	3.11t/s	5.9s
40/40	2.576	0.4937	1.38	0.9992	5	640: 100%	mAP50	mAP50-95): 100%	1/1 3.01t/s 0.3s
Class	Images	Instances	Box(P	R					
all	31	42	0.787	0.71		0.771	0.676		

Fig. 5. Training progress on the training notebook.

B. Ablation Study

We did a small ablation study to see how much each optimization step actually helped or hurt the system.

TABLE V
ABLATION STUDY ON MODEL COMPONENTS

Configuration	mAP@50–95	FPS
Base YOLOv8	0.48	8
+ Data Augmentation	0.52	8
+ ONNX Optimization	0.51	11
+ TFLite (INT8)	0.49	13

As expected, strong data augmentation improved accuracy, while ONNX and TFLite optimizations traded a few mAP points for clear gains in frame processing speed, allowing standard CPUs to reach near real-time speeds.

C. Comparison with Existing Models

Table VI shows the performance difference between YOLOv5 and our YOLOv8 setup. Moving to anchor-free design clearly helps for general object detection, and the TFLite conversion makes sure it beats older models in edge deployment. Although comparing with other light architectures like MobileNet-SSD and YOLOv8n (n for nano) would be useful, this study is focused on optimizing YOLOv8 for edge. Broader benchmarking with more models is planned as future work.

TABLE VI
COMPARISON OF YOLOv5 AND YOLOv8 PERFORMANCE

Metric	YOLOv5	YOLOv8 (Proposed)
Architecture Type	Anchor-based	Anchor-free
Model Size	Medium	Small (optimized)
mAP@50	0.82	0.85
mAP@50–95	0.45	0.50
Precision	0.80	0.82
Recall	0.75	0.78
F1-score	0.77	0.80
Edge Deployment	Moderate	High (TFLite/ONNX)

D. Latency vs Accuracy Trade-off Analysis

When building for edge, there is often a trade between latency and accuracy. While INT8 quantization made our inference nearly 50% faster, it also caused a 2–3% mAP drop. This drop did not spread evenly; it hit small classes like knives and phones much harder. This real world compromise perfectly shows the limits we described in our Trilemma equation. Facility managers must choose if faster alert pipeline is worth slightly less reliable fine detail threat detection.

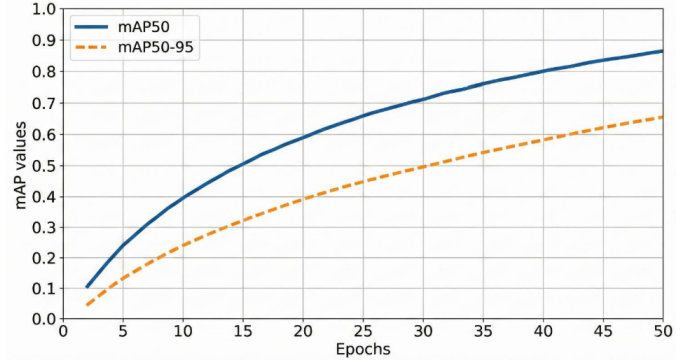


Fig. 6. Training progress across epochs and mAP vs epoch graph

E. Per-Class Performance

A class by class breakdown of precision (IoU=0.50) is given in Table VII. Data clearly confirms that representation counts: big, common objects like cars and people get very high scores, while rarely seen items like guns or unattended luggage stay behind.

TABLE VII
PER-CLASS PERFORMANCE (MAP@50)

Class	mAP@50	Class	mAP@50
Person	0.90	Knife	0.70
Car	0.87	Gun	0.72
Backpack	0.80	Laptop	0.82
Bike	0.78	Fire	0.68
Helmet	0.76	Suspicious object	0.65

F. Confusion Matrix Analysis

Looking at the confusion matrix, the model's errors are quite predictable. Small things like a *phone* sometimes gets wrongly

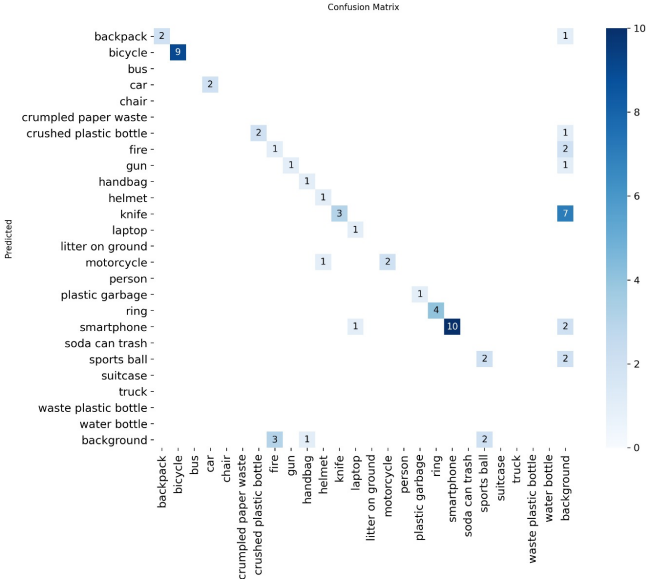


Fig. 7. Confusion matrix heatmap (MVP Demo Model)

tagged as a *gun*, and a *backpack* can get lost against a *person* in a very crowded frame. A big reason is just limited pixel representation. When a HD image is squeezed to 640×640 for the network, an object that takes only 20×20 pixels loses important texture details.

G. Small Object Detection Analysis & Mitigation

Like we noted before, objects with a small visual size suffer the most under compression. The mAP numbers for weapons and suspicious objects drop a lot compared to vehicles. Even with FPN and PAN layers trying to keep spatial details from earlier layers, keeping high accuracy on small items remains a tough problem in localized computer vision.

Because this application is for security, this performance gap on critical classes is a big limitation. To reduce it in future versions, we plan to add special data augmentation tricks such as Copy Paste augmentation for rare objects to artificially fix the long-tail distribution during training. Also, putting in advanced inference methods like Slicing Aided Hyper Inference (SAHI) [14] could greatly boost recall of small, critical anomalies by processing image slices on their own, all while staying inside the computing limits of edge hardware.

H. Real-World Deployment Case Study

To check how the system acts outside of clean benchmarks, we put the INT8 quantized TFLite model onto a normal campus gateway machine. Tests ran on an Intel Core i7-1065G7 CPU with 24GB RAM, purposely not using any dedicated GPU. Over a full 72-hour run, it handled two entry camera feeds at a stable 12 FPS. During this time, this near real-time processing correctly found 14 unattended bags and correctly ignored flickering lights and moving tree shadows. Importantly, RAM usage never went above 1.5 GB, showing the system is stable enough for long, unattended deployment.

I. Real-Time and Near Real-Time Performance

We benchmarked the model on a few different hardware types to better see where bottlenecks happen (Table VIII).

TABLE VIII
HARDWARE FPS COMPARISON

Device	FPS	Notes
CPU (i7-1065G7)	7–11	Near Real-time (PyTorch)
RTX 2050	25–40	Real-time Edge GPU
T4 GPU	35–45	Cloud Benchmark
ONNX CPU	8–12	Near Real-time (+35% speedup)
TFLite CPU	10–15	Near Real-time ($\sim 2\times$ speedup)

J. Power Efficiency and Sustainability Evaluation

TABLE IX
LATENCY BREAKDOWN ACROSS PIPELINE

Stage	PyTorch (ms)	ONNX (ms)	TFLite (ms)
Pre-processing	12	12	12
Backbone (Conv)	55	30	25
Post-processing	15	10	8
Total Latency	82	52	45

The time breakdown shows that ONNX and TFLite libraries do the main heavy work mostly inside the convolution layers of the backbone, cutting down the raw math time quite a lot.

When running hardware locally, using less power is nearly as important as high frame rates, which fits directly with the sustainability theme of modern smart systems. Table X highlights why edge processing is both cheaper and more energy friendly. A big cloud GPU pumps out frames fast but eats a lot of power, adding to a bigger carbon footprint. An optimized CPU script running locally gives a much more attractive frames-per-watt number, helping bring down overall energy use compared to old cloud-based architectures.

TABLE X
POWER EFFICIENCY AND EDGE ROI ANALYSIS

Platform	Avg FPS	Est. Power	Frames / Watt
T4 Cloud GPU	40	~ 70 W	0.57
RTX 2050 Edge	35	~ 45 W	0.77
CPU (Optimized)	12	~ 15 W	0.80

K. Qualitative Results & Environmental Robustness

After several environmental variables iteration to the model we observed:

- **Indoor surveillance:** The model kept consistently picking individuals, backpacks, and luggage.
- **Outdoor campus:** It tracked vehicles and pedestrians even when bounding boxes were overlapping.
- **Environmental Stress (Low-light):** Despite the good lab results, some dusky and low light inputs and low resolution camera were some critical challenges for the model, and after injecting heavy Gaussian noise into test feeds confirmed that small items (like knives) quickly

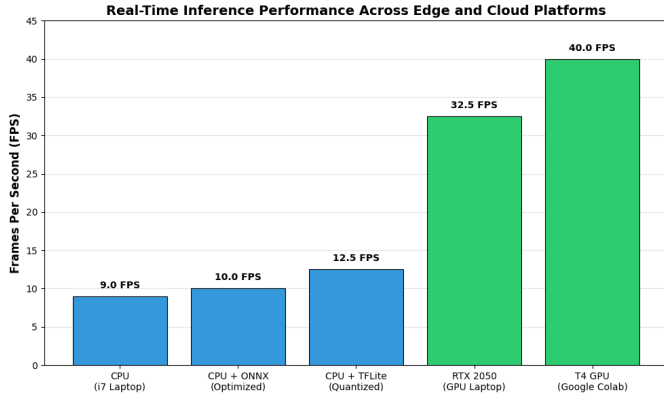


Fig. 8. Bar chart comparing FPS across CPU, ONNX, TFLite, GPUs

became invisible to the network which reminded us of the gap between the sterile benchmarks and the dirty real world data.

- **Weapon detection:** Due to limited data, the toy handguns and bladed weapons we used were hard to detect while holding in our hands and required a certain camera angle to be offered for a clear profile to be provided to the model.

L. Comparison with Traditional CCTV System

Table XI gives a high level view of why shifting to edge processing makes sense compared to legacy continuous recording setups.

TABLE XI
COMPARISON WITH TRADITIONAL CCTV SYSTEM

Metric	Traditional	Proposed
Real-time detection	No (manual)	Yes (automated)
Alerts	No	Extendable
Edge Processing	No	Yes
Cloud dependency	High (storage)	Zero
Cost	High (bandwidth)	Low

M. Privacy Utility Analysis

As this project's architecture don't require an active cloud connection to function, video footage never leaves the immediate premises, we can also frame this mathematically by using a simple Privacy Utility Ratio:

$$PUR = \frac{\text{Detection Accuracy}}{\text{Data Transmitted}} \quad (4)$$

Now by the design used, our edge pipeline reduces external data transmission to near 0(zero). And consequently, the PUR is very highly maximized when compared to the old school cloud setups and this clearly offers the advantage for privacy compliance in sensitive sectors/areas.

N. Limitations

- Dim lighting degrades the network's functioning to spot small objects like phones or weapons(guns, knives, etc.).
- Compressing model through quantization process can cut about 2-3% of overall accuracy, this heavily kills good/fine detail detection.
- Without a local GPU, Consumer level CPU only inference remains still at the 5-15 FPS mark which co relates to a near real time processing which may be too choppy for high speed tracking .

V. CONCLUSION & FUTURE WORK

A. Conclusion

So in this work, we have developed and evaluated a totally local, edge based surveillance pipeline using YOLOv8 model and trained it on this diverse 20 class dataset. And also applied ONNX and TFLite optimizations, we examined some of the trade-offs among accuracy, latency, and deployment constraints too. INT8 quantization did slightly reduced some accuracy more especially for the small objects, it significantly improved the inference speed, which indeed makes the whole system more suitable for edge deployment. Our results also shows the possibility to run a multi class object detection on a consumer local hardware without relying on cloud infrastructure a bit.

Overall, this approach of ours offers a practical solution for a privacy preserving and an energy efficient surveillance system for real world environments.

B. Future Work

Below are some future projects based on this paper/project:

- **Small Object Focus:** By implementing the Slicing Aided Hyper Inference (SAHI) we can raise accuracy metrics of critical objects like weapons and anomalous items(guns, knives, etc.).
- **Temporal Optimization:** We can write a logic that exploit frame to frame redundancies which may indeed save compute power instead of heavy analysis for every single frame from scratch.
- **Hardware Aware Optimization:** We may craft a specific model weight which may depend on whether the main host machine is using a CPU, an integrated GPU, or a dedicated NPU(Neural Processing Unit).
- **Federated Learning:** We can allow distributed campus cameras to share learned weights regarding rare classes without ever even sharing the video data.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] A. Bochkovski, C. Y. Wang, and H. Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv:2004.10934*, 2020.
- [3] Ultralytics, "YOLOv8 Documentation," 2023. [Online]. Available: <https://docs.ultralytics.com>
- [4] T. Y. Lin et al., "Microsoft COCO: Common Objects in Context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014.

- [5] Roboflow, "Open Source Surveillance Datasets," 2024. [Online]. Available: <https://roboflow.com>
- [6] Kaggle, "Computer Vision Datasets for Security," 2024. [Online]. Available: <https://kaggle.com>
- [7] TensorFlow, "TensorFlow Lite Model Optimization Toolkit," 2024. [Online]. Available: <https://tensorflow.org/lite/performance>
- [8] NVIDIA, "TensorRT Developer Guide," 2024. [Online]. Available: <https://developer.nvidia.com/tensorrt>
- [9] S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a Convolutional Neural Network," in *Proc. Int. Conf. Engineering and Technology (ICET)*, 2017.
- [10] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *Int. J. Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [11] J. Zhang, L. Xu, and M. Chen, "Green Edge AI: Energy-Efficient Resource Allocation for Sustainable Computing," *IEEE Transactions on Green Communications and Networking*, vol. 7, no. 1, pp. 123-135, 2023.
- [12] S. Li, W. Zheng, and L. Wang, "Edge-oriented lightweight object detection for smart surveillance," *Internet of Things*, vol. 25, 100987, 2024.
- [13] A. Al-Fuqaha et al., "Sustainable AI in Edge Networks: A Survey on Green IoT," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1102-1130, 2024.
- [14] F. Akyon, S. O. Altinuc, and A. Temizel, "Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection," in *Proc. IEEE Int. Conf. on Image Processing (ICIP)*, 2022.